

From Enock (data / RAG retrieve) to Alex (agents)

What is already in Supabase, what the code does, and the first steps to plug in the multi-agent layer.

Prepared by Enock Katenda · For Alex Ji · cc Leandro Ardiles, Tafadzwa Nyemba · 5 October 2026

1. One-sentence status

The court-opinion corpus is ingested into one shared Supabase Postgres database, identified by **LEXIS citation** (not file name), grouped into **litigation families**, chunked, and searchable with **pgvector**. Enock's track is the RAG retrieve store. Alex's track starts here: generate answers with Groq agents using this data. Do not rebuild ingest. Do not use the Capstone I SQLite app as the corpus.

2. What Enock finished

Item	Status	Where
Word → structured decisions (rules, not Groq)	Done	<code>backend/app/ingest/</code>
Identity = LEXIS cite; (2) files are other decisions in the same lawsuit	Done	<code>decisions.lexis_citation</code>
Litigation families + prior/subsequent links	Done	<code>litigation_families</code> , <code>decision_relations</code>
Load full corpus (not only the pilot)	Done	540 decisions, 427 families, 383 relations
Chunk + local embeddings + pgvector index	Done	14,987 chunks, 0 offset mismatches
Retrieve CLI (search, not a legal answer)	Done	<code>python -m app.ingest.load_supabase --query "..."</code>
50-decision frozen pilot for Q1/Q2	Done, pending Tafadzwa labels	<code>backend/data/pilot_50_manifest.json</code>
HTTP retrieve API for agents	Not built. Python <code>retrieve()</code> now returns a list of dicts; CLI still prints.	<code>retrieve()</code> in <code>load_supabase.py</code>
Multi-agent readers / adjudicator / Groq	Alex	—

No Groq is used in ingest. Parsing is rule-based. Chunking is character windows. Embeddings are a local model (BAAI/bge-small-en-v1.5 , 384-d) on the laptop. The only cloud write is Supabase. Save the Groq budget for agent runs.

3. How to think about the data (Leandro's rules)

- **Unit of identity:** LEXIS citation, e.g. 2020 U.S. Dist. LEXIS 187996 .
- Craig v. Simon.docx and Craig v. Simon(2).docx are **different decisions** (8th Cir. vs D. Minn.), same **family**.
- Do not collapse filenames. Do not key experiments on short_name alone.
- About 86 files have no U.S. ... LEXIS line (often state courts) — lexis_citation is null; they still have source_filename .
- One true re-export pair shares 2021 U.S. LEXIS 1197 (two captions, one cite). Only one row was loaded.

4. Supabase tables

Project: <https://mvpecqzquuxpwuazyq.supabase.co> · Dashboard → Table Editor.

Table	Rows	Alex uses it for
decisions	540	Full opinion_text / dissent / concur, cite, court, date, family_id
litigation_families	427	Related district + appeal decisions
decision_relations	383	Prior / affirmed / related cites from LEXIS headers
opinion_chunks	14,987	Retrieval: text , char_start , char_end , embedding , section

Offsets are into that section's stored text: opinion_text[char_start:char_end] == chunk.text .

5. Baby steps for Alex

1. **Do not start a new repo.** Wait for Enock's ingest branch on github.com/leanardiles/us-court-decisions-analyzer (this code is not on GitHub until Enock pushes).
2. **Get Supabase access** via dashboard invite (not a password in Slack). Confirm you can open decisions and opinion_chunks .
3. **Local setup** from backend/ :

```
python3 -m venv venv && source venv/bin/activate && pip install -r requirements.txt
```


Copy .env.example → .env . Enock will give URL + DB password privately. This Mac has no python binary; use ./venv/bin/python or activate the venv.
4. **Smoke-test retrieve** (search, not an LLM answer):

```
./venv/bin/python -m app.ingest.load_supabase --query "Purcell principle days before an election" --k 5
```


Restrict to one decision: add --lexis "2024 U.S. Dist. LEXIS 185656"

5. **Read a full opinion for an agent.** SQL: `select lexis_citation, source_filename, opinion_text from decisions where lexis_citation = '...' .` The 50 were *chosen* as 2k–50k characters; full text is stored. About 9 of 50 have no LEXIS cite — use `source_filename` . The other 490 can be huge — use `retrieve` for those, not the whole blob.
6. **Plug retrieve into System B, do not replace ingest.** From Python:

```
from app.ingest.load_supabase import retrieve
```

```
hits = retrieve(question, k=8, lexis_citation=cite, print_results=False)
```

Each hit has `text` , `char_start` , `char_end` , `section` , `score` , `lexis_citation` , `source_filename` . Or wrap that in a FastAPI route later. Prompt = question + retrieved chunks + (optional) full opinion for short cases.
7. **Start on the frozen 50, not all 540.** Manifest: `backend/data/pilot_50_manifest.json` . Q1 = election type (control). Q2 = days between trial ruling and election (known failure). Tafadzwa owns ground truth keyed by LEXIS citation.
8. **Run System A vs System B on that same 50.** System A = one Groq call. System B = two independent readers + adjudicator; preserve disagreement. Models from the 23 Sep shortlist (GPT-OSS 20B / 120B, Qwen 3.8 27B). Log case id, model, role, prompt version, answer, evidence spans, confidence, tokens, latency, cost.
9. **Do not write agent answers into `opinion_text` .** Add new tables later (e.g. `agent_runs`) or keep JSONL experiment logs first.
0. **Human stays last.** Agents flag; validators/scholar decide. That is unchanged from Capstone I.

6. Enock's way forward (do this next)

Yes — you should push the code to GitHub. Alex cannot continue from this laptop. The live data is already in Supabase; GitHub is for the ingest/retrieve *code*. Do it as a PR or a branch. Ask Leandro if you cannot push to `main` (the remote is his: `leanardiles/us-court-decisions-analyzer`).

Push

- `backend/app/ingest/` , `supabase/` , `.env.example` , `backend/app/core/config.py` , `.gitignore` , `pilot_50_manifest.json`
- This handover PDF

Do not push

- `backend/.env` (password + keys) — already gitignored
- The 541 Word files (optional; they are already in Supabase). If the folder is huge, leave it local or gitignore it.
- `backend/data/pilot_50_decisions.json` (full opinion text dump)

Then

1. Open a PR (or ask Leandro to merge) so Alex can clone the ingest code.
2. Invite Alex (and Tafadzwa) to the Supabase project as team members.
3. Send Alex the DB password in a private channel, not in this PDF and not in the group chat.

4. Give Tafadzwa `pilot_50_manifest.json` and ask for Q1/Q2 labels keyed by LEXIS citation.
5. Stay owner of ingest/retrieve. If Alex needs an HTTP `/retrieve` route, that is a small follow-up on your side — not a blocker for him to start in Python.

7. Commands cheat sheet

Task	Command (from <code>backend/</code>)
Search chunks	<code>./venv/bin/python -m app.ingest.load_supabase --query "..." --k 5</code>
Search one case	<code>same + --lexis "2020 U.S. Dist. LEXIS 187996"</code>
Re-parse Word → DB	<code>./venv/bin/python -m app.ingest.load_supabase --all</code>
Rebuild embeddings	<code>./venv/bin/python -m app.ingest.load_supabase --embed</code> (slow; hours on CPU)
Schema	<code>backend/app/ingest/supabase_schema.sql</code>

8. What not to do

- Do not stand up a second database or a chatbot UI.
- Do not send 540 full opinions through Groq.
- Do not treat filename as the primary key.
- Do not commit `.env`.
- Do not silently replace the 50-case manifest.
- Do not connect to `db.*.supabase.co` (IPv4 will fail). Use the session pooler in the appendix.
- Do not query these tables with the publishable REST key.

9. Appendix — connect, SQL, retrieve contract

Read this before the first agent run. Passwords stay out of this file; Enock sends them privately.

9.1 Connect (session pooler, not the “direct” host)

db.mvpecqzquuxpwuazyaq.supabase.co:5432 does not resolve on a typical IPv4 laptop. Use:

- Host `aws-0-us-east-2.pooler.supabase.com` · port `6543` · db `postgres`
- User `postgres.mvpecqzquuxpwuazyaq` · SSL required
- `SUPABASE_URL=https://mvpecqzquuxpwuazyaq.supabase.co`
- `SUPABASE_DB_PASSWORD='...'` — **single-quote** it (# and \$)
- `SUPABASE_DB_URL` = same user @ pooler host, URL-encoded password, `?sslmode=require`

9.2 Embedding contract (must match)

- Model `BAAI/bge-small-en-v1.5` · 384-d · local `fastembed`
- Distance `cosine` · Postgres `<=>`
- Chunks `~3200` chars / 400 overlap · section = `opinion | dissent | concur`
- Query prefix is already applied in `embed_query()`
- First run downloads the model from Hugging Face (no token)

The publishable `sb_publishable_...` key cannot REST-query these tables (“Secret API key required”). Use Postgres (password / pooler), not `supabase-js` + anon, until RLS is added.

9.3 SQL Alex will actually run

```
-- one decision; if lexis_citation is null, key by source_filename
select lexis_citation, source_filename, court, decision_date, opinion_text, dissent_text
from decisions where lexis_citation = '2020 U.S. Dist. LEXIS 187996';
select * from decisions where source_filename = 'Abdus-Sabur v. Evans.docx';
-- siblings in the same lawsuit
select d.short_name, d.lexis_citation, d.court_level, d.decision_date, d.source_filename
from decisions d join litigation_families f on f.id = d.family_id
where f.family_key = 'craig v simon' order by d.decision_date;
```

Pilot 50: `backend/data/pilot_50_manifest.json`. About 9 of 50 have no U.S. LEXIS cite — key those by `source_filename`. Chosen as 2k–50k chars, max 3 per family. Full opinion text is stored (the length cap was only for picking the 50).

9.4 Python retrieve (returns dicts; CLI still prints)

```
from app.ingest.load_supabase import retrieve
hits = retrieve("days between ruling and election", k=8,
               lexis_citation="2020 U.S. Dist. LEXIS 187996", print_results=False)
# keys: decision_id, short_name, lexis_citation, source_filename,
#       section, chunk_index, char_start, char_end, score, text
```

9.5 Experiment log (JSONL now, `agent_runs` table later)

One object per agent call: `lexis_citation` or `source_filename`, `system` (A|B), `agent_role` (`reader_a|reader_b|adjudicator`), `model`, `prompt_version`, `question_id` (Q1|Q2), `answer`, `cited_char_start`, `cited_char_end`, `confidence`, `agree`, `tokens_in`, `tokens_out`, `latency_ms`, `cost_usd`, `run_at`. Do not write these into `decisions.opinion_text`.

9.6 Who owns what

Enock ingest / retrieve / schema · **Alex** Groq readers + adjudicator on the frozen 50 · **Leandro** GitHub + official Groq account (Capstone I app is not the corpus) · **Tafadzwa** Q1/Q2 labels keyed by LEXIS (or filename if NO CITE).

Repo (after push): `github.com/leanardiles/us-court-decisions-analyzer` · Code: `backend/app/ingest/` · Store: Supabase project `mvpecqzquuxpwuazyaq` · Experiment design remains the 23 September status report (System A vs System B on a frozen set).